INFORME PRÁCTICA 3
PROCESAMIENTO DE IMÁGENES Y EXTRACCIÓN DE CARACTERÍSTICAS

Andrés Gil Vicente & Jorge Carnicero Príncipe

4 de noviembre de 2025

Índice

1. Introducción **4**

2. Preparación del Entorno de Trabajo **4**

2.1. Carga de Librerías Necesarias 4

2.2. Funciones Auxiliares 5

3. Parte A: Filtro Gaussiano y Detección de Bordes **5**

3.1. Tarea A.1: Implementación del Filtro Gaussiano 5

3.1.1. Explicación del Filtro Gaussiano 6

3.2. Tarea A.2: Aplicación del Filtro Gaussiano 7

3.3. Tarea A.3: Implementación del Detector de Bordes de Sobel 7

3.3.1. Explicación del Detector de Sobel 8

3.4. Tarea A.4: Aplicación del Detector de Sobel 9

3.4.1. Constante de Estandarización 9

3.5. Tarea A.5: Implementación del Detector de Bordes de Canny 10

3.5.1. Explicación del Algoritmo de Canny 11

3.6. Tarea A.6: Aplicación del Detector de Canny 11

3.6.1. Constante de Estandarización para Canny 12

4. Preguntas Adicionales de la Parte A **13**

4.1. Pregunta A.1: Efecto del Ruido en la Detección de Bordes 13

4.1.1. Comparación con y sin Filtro Gaussiano 14

4.1.2. Análisis de Resultados 15

4.1.3. Comparaciones adicionales 15

4.2. Pregunta A.2: Comparación con scikit-image 17

4.2.1. Análisis Comparativo 18

5. Parte B: Operadores Morfológicos **20**

5.1. Tarea B.1: Implementación del Método de Binarización 20

5.1.1. Explicación del Proceso de Binarización 21

5.2. Tarea B.2: Implementación del Método de Dilatación 21
5.2.1. Explicación del Proceso de Dilatación 23
5.3. Tarea B.3: Implementación del Método de Erosión 23
5.3.1. Explicación del Proceso de Erosión 25
5.4. Pregunta B.1: Aplicación de los Operadores Morfológicos 25
5.4.1. Análisis de Resultados 26
5.4.2. Aplicaciones Prácticas de los Operadores Morfológicos 28

6. Parte C: Detección de Esquinas 28
6.1. Tarea C.1: Creación de Carpeta para Resultados 28
6.2. Tarea C.2: Métodos de Carga de Imágenes 28
6.2.1. Análisis de Diferencia entre Métodos de Carga 29
6.3. Tarea C.3: Implementación del Detector de Esquinas de Harris 30
6.4. Tarea C.4: Aplicación del Detector de Harris 31
6.5. Tarea C.5: Implementación del Detector de Esquinas de Shi-Tomasi 32
6.6. Tarea C.6: Aplicación del Detector de Shi-Tomasi 33
6.7. Pregunta C.1: Detección de Esquinas con Harris en Sudoku y Tennis 34
6.7.1. Definición de Parámetros 34
6.7.2. Aplicación a las Imágenes 35
6.8. Pregunta C.2: Detección de Esquinas con Shi-Tomasi en Sudoku y Tennis . . 36
6.8.1. Definición de Parámetros 36
6.8.2. Aplicación a las Imágenes 37
6.9. Comparación entre Harris y Shi-Tomasi 38

7. Conclusión General 40
7.1. Logros Principales 40
7.2. Conclusiones Técnicas 40

1. Introducción

En esta tercera práctica de Computer Vision, nos centramos en varios aspectos fundamentales del procesamiento de imágenes: el filtrado gaussiano, la detección de bordes, el uso de operadores morfológicos y la detección de esquinas. Estos conceptos son esenciales en el procesamiento digital de imágenes, ya que permiten mejorar la calidad de las imágenes y extraer información relevante sobre sus características estructurales.

El objetivo principal de esta práctica es implementar y comparar diferentes métodos de detección de bordes (Sobel y Canny) y esquinas (Harris y Shi-Tomasi), aplicándolos a un conjunto de imágenes de prueba. Además, se estudiará el efecto del ruido en estos algoritmos y se compararán los resultados con implementaciones de librerías especializadas como scikit-image.

2. Preparación del Entorno de Trabajo

2.1. Carga de Librerías Necesarias

Para el desarrollo de esta práctica, utilizamos las siguientes librerías de Python:

Código 2.0: Importación de librerías

```
import cv2
from typing import List, Optional
import numpy as np
import imageio
import copy
import glob
import matplotlib.pyplot as plt
from utils import *
```

Las librerías utilizadas cumplen las siguientes funciones:

- **cv2**: OpenCV para procesamiento de imágenes y visión por computador
- **numpy**: Manipulación eficiente de arrays y operaciones matriciales
- **imageio**: Lectura y escritura de imágenes
- **copy**: Creación de copias de objetos
- **glob**: Búsqueda de archivos mediante patrones
- **matplotlib**: Visualización de imágenes y gráficos

Además, del fichero `utils.py` importamos todas las funciones que ya tiene definidas, puesto que nos serán de ayuda durante la práctica. Por ejemplo, la función de `non_max_suppression`.

2.2. Funciones Auxiliares

Definimos algunas funciones auxiliares que nos facilitarán el trabajo durante la práctica:

Código 2.1: Funciones auxiliares para visualización

```python
def show_image(img: np.array, img_name: str = "Image"):
    cv2.imshow(img_name, img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

def load_images(filenames: List) -> List:
    return [cv2.imread(filename) for filename in filenames]
```

La función `show_image` nos permite visualizar imágenes de forma rápida durante el desarrollo, mientras que `load_images` facilita la carga de múltiples imágenes desde disco.

3. Parte A: Filtro Gaussiano y Detección de Bordes

El objetivo de este apartado es detectar los bordes de las imágenes de la carpeta ‘‘`data/partA-B`’’ mediante diferentes métodos. El proceso se divide en las siguientes tareas:

3.1. Tarea A.1: Implementación del Filtro Gaussiano

El filtro gaussiano es fundamental para eliminar ruido en las imágenes antes de aplicar operadores de detección de bordes. Implementamos la función `gaussian_blur`:

Código 3.0: Implementación del filtro gaussiano

```python
def gaussian_blur(img: np.array, sigma: float, filter_shape: Optional[List]
↪  = None, verbose: bool = False) -> np.array:

    # Para evitar aplicar ningún filtro gaussiano (útil para comparar)
    if sigma == 0:
        if verbose:
            show_image(img=img, img_name=f"Gaussian Blur: Sigma = {sigma}")

        return None, img.astype(np.uint8)
```

```
    else:
        # Cálculo del tamaño del kernel si no se proporciona
        if filter_shape == None:
            k = max(3, int(6*sigma + 1))
            if k % 2 == 0:
                k += 1  # Aseguramos que sea impar
            kh,kw = k,k
        else:
            kh, kw = int(filter_shape[0]), int(filter_shape[1])
            if kh // 2 == 1: kh +=1
            if kw // 2 == 1: kw += 1

        # Creación de las matrices de coordenadas del filtro
        cy,cx = kh // 2, kw // 2
        y, x = np.mgrid[-cy:kh-cy, -cx:kw-cx]

        # Definición de la fórmula gaussiana
        formula = np.exp(-(x**2+y**2)/(2*sigma**2))

        # Normalización del filtro
        gaussian_filter = (formula / formula.sum()).astype(np.float32)

        # Aplicación del filtro a la imagen
        gb_img = cv2.filter2D(img, -1, gaussian_filter)

        if verbose:
            show_image(img=gb_img, img_name=f"Gaussian Blur: Sigma =
            ↪  {sigma}")

        return gaussian_filter, gb_img.astype(np.uint8)
```

3.1.1. Explicación del Filtro Gaussiano

El filtro gaussiano implementado sigue estos pasos:

1. **Cálculo del tamaño del kernel**: Si no se proporciona un tamaño específico, se calcula como $k = \text{máx}(3, 6\sigma + 1)$, asegurando que dicho tamaño sea impar.
2. **Generación de coordenadas**: Se crea una malla de coordenadas centrada en el kernel.
3. **Aplicación de la fórmula de la distribución gaussiana**: $G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$
4. **Normalización**: Se divide por la suma total para que la suma del filtro sea 1, evitando cambios en el brillo global.

5. **Convolución**: Se aplica el filtro creado a la imagen usando `cv2.filter2D`.

3.2. Tarea A.2: Aplicación del Filtro Gaussiano

Aplicamos el filtro gaussiano a todas las imágenes de la carpeta ‘‘`data/partA-B`’’:

Código 3.1: Carga y procesamiento de imágenes

```
imgs_gen_path = '../data/partA-B/*.jpg'
imgs_path = sorted(glob.glob(imgs_gen_path))
imgs = load_images(imgs_path)

# Aplicación del filtro gaussiano
gauss_sigma = 1.4   # Valor moderado que aplica blur sin perder mucha
↪  información
gb_imgs = [gaussian_blur(img,gauss_sigma,None,False)[1] for img in imgs]
```

El valor de $\sigma = 1{,}4$ se eligió porque es el utilizado en los ejemplos de las diapositivas del tema 3, proporcionando un buen equilibrio entre eliminación de ruido y preservación de detalles.



Figura 1: Comparación entre imagen original (izquierda) y con filtro gaussiano aplicado (derecha)

3.3. Tarea A.3: Implementación del Detector de Bordes de Sobel

El operador de Sobel calcula el gradiente de la imagen en las direcciones horizontal y vertical. Implementamos la función `sobel_edge_detector`:

Código 3.2: Implementación del detector de Sobel

```
def sobel_edge_detector(img: np.array, filter: np.array, gauss_sigma: float,
                        gauss_filter_shape: Optional[List] = None, verbose:
                        ↪  bool = False) -> np.array:
```

```
    # Conversión a escala de grises
    gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Aplicación de filtro gaussiano para mejorar detección de bordes
    blurred = gaussian_blur(img=gray_img, sigma=gauss_sigma,
                            filter_shape=gauss_filter_shape,
                             verbose=verbose)[1]

    # Re-escalado para trabajar con valores normalizados
    blurred = blurred/255

    # Detección de bordes verticales
    v_edges = cv2.filter2D(blurred, -1, filter)

    # Transformación del filtro para bordes horizontales
    filter = filter.T

    # Detección de bordes horizontales
    h_edges = cv2.filter2D(blurred, -1, filter)

    # Cálculo de la magnitud del gradiente
    sobel_edges_img = np.hypot(v_edges, h_edges)

    # Cálculo del ángulo del gradiente
    theta = np.arctan2(h_edges, v_edges)

    if verbose:
        show_image(img=sobel_edges_img, img_name="Sobel Edges")

    return np.squeeze(sobel_edges_img), np.squeeze(theta)
```

3.3.1. Explicación del Detector de Sobel

El proceso implementado incluye:

1. **Conversión a escala de grises**: Los operadores de bordes trabajan sobre la intensidad de la imagen.
2. **Suavizado gaussiano**: Fundamental para reducir el ruido que puede generar falsos bordes.
3. **Cálculo de gradientes**: Se aplican los kernels de Sobel en direcciones horizontal y vertical.
4. **Magnitud del gradiente**: Se calcula como $\sqrt{G_x^2 + G_y^2}$.

5. **Dirección del gradiente**: $\theta = \arctan 2(G_y, G_x)$.

3.4. Tarea A.4: Aplicación del Detector de Sobel

Aplicamos el detector de Sobel a todas las imágenes:

Código 3.3: Aplicación del detector de Sobel

```
gauss_sigma = 1.4
sobel_filter = np.array([[-1, 0, 1],
                         [-2, 0, 2],
                         [-1, 0, 1]], dtype=np.float32)

sobel_edges_imgs = [sobel_edge_detector(img=img, filter=sobel_filter,
                                        gauss_sigma=gauss_sigma,
                                        gauss_filter_shape=None,
                                        ↪  verbose=False)[0]
                    for img in imgs]
```

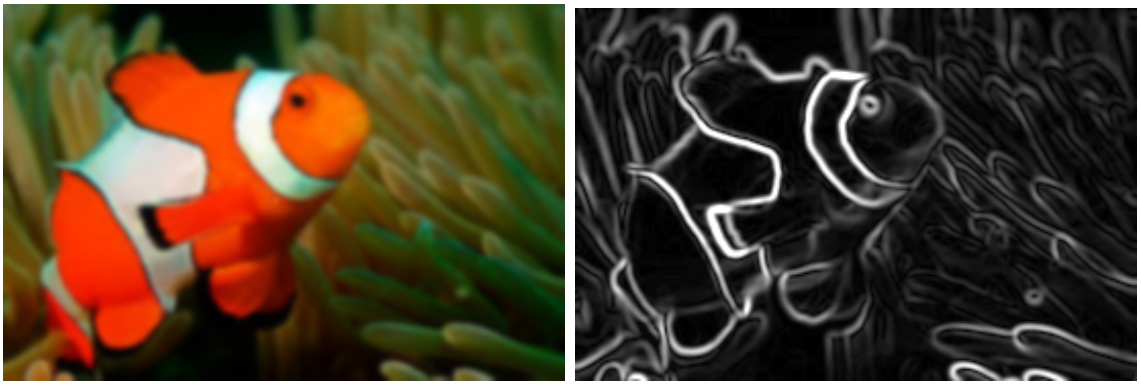


Figura 2: Imagen con filtro gaussiano (izquierda) y bordes detectados con Sobel (derecha)

3.4.1. Constante de Estandarización

En las diapositivas teóricas se menciona la necesidad de estandarizar el filtro de Sobel multiplicando por 1/8. Por ello, y con el objetivo de comprender mejor el funcionamiento de nuestro detector, decidimos implementar esta variante adicional:

Código 3.4: Sobel con constante de estandarización

```
sobel_filter_standard = (1/8)*np.array([[-1, 0, 1],
                         [-2, 0, 2],
                         [-1, 0, 1]], dtype=np.float32)
```

```
sobel_edges_imgs_standard = [sobel_edge_detector(
                                img=img, filter=sobel_filter_standard,
                                gauss_sigma=gauss_sigma,
                                gauss_filter_shape=None,
                                verbose=True)[0]
    for img in imgs]
```

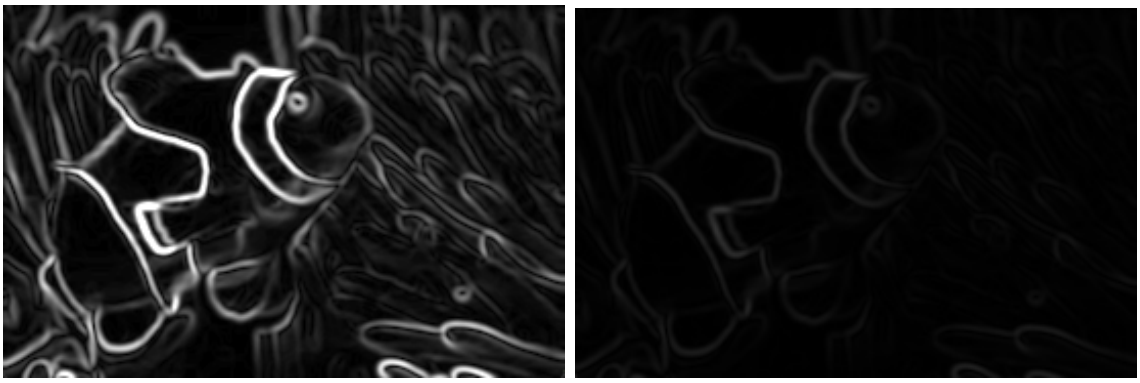


Figura 3: Imagen con sobel (izquierda) y sobel estandarizado (derecha)

Como vemos en las imágenes, la estandarización produce bordes con intensidades menos marcadas por el factor de estandarización.

3.5. Tarea A.5: Implementación del Detector de Bordes de Canny

El algoritmo de Canny es más complejo y robusto que Sobel, incluyendo supresión de no máximos y umbralización con histéresis. Implementamos `canny_edge_detector`:

Código 3.5: Implementación del detector de Canny

```
def canny_edge_detector(img: np.array,
                        sobel_filter: np.array,
                        gauss_sigma: float,
                        gauss_filter_shape: Optional[List] = None,
                        verbose: bool = False):

    # Detección de bordes con Sobel
    sobel_edges_img, theta = sobel_edge_detector(
                                img=img,
                                filter=sobel_filter,
                                gauss_sigma=gauss_sigma,
                            gauss_filter_shape=gauss_filter_shape,
```

```
                                verbose=verbose)



    # Aplicación de supresión de no máximos
    canny_edges_img = non_max_suppression(
                    img=sobel_edges_img, theta=theta
                )

    if verbose:
        show_image(canny_edges_img, img_name="Canny Edges")

    return canny_edges_img
```

3.5.1. Explicación del Algoritmo de Canny

El algoritmo de Canny mejora la detección de Sobel mediante:

- **Supresión de no máximos**: Solo se mantienen los píxeles realmente son máximos locales en la dirección del gradiente. Descartando los "no tan máximos".

3.6. Tarea A.6: Aplicación del Detector de Canny

Aplicamos el detector de Canny a todas las imágenes:

Código 3.6: Aplicación del detector de Canny

```
sobel_filter = np.array([[-1, 0, 1],
                         [-2, 0, 2],
                         [-1, 0, 1]], dtype=np.float32)

gauss_sigma = 1.4
gauss_filter_shape = None

canny_edges = [canny_edge_detector(
                    img=img,
                    sobel_filter=sobel_filter,
                    gauss_sigma=gauss_sigma,
                    gauss_filter_shape=gauss_filter_shape,  verbose=True)
for img in imgs]
```

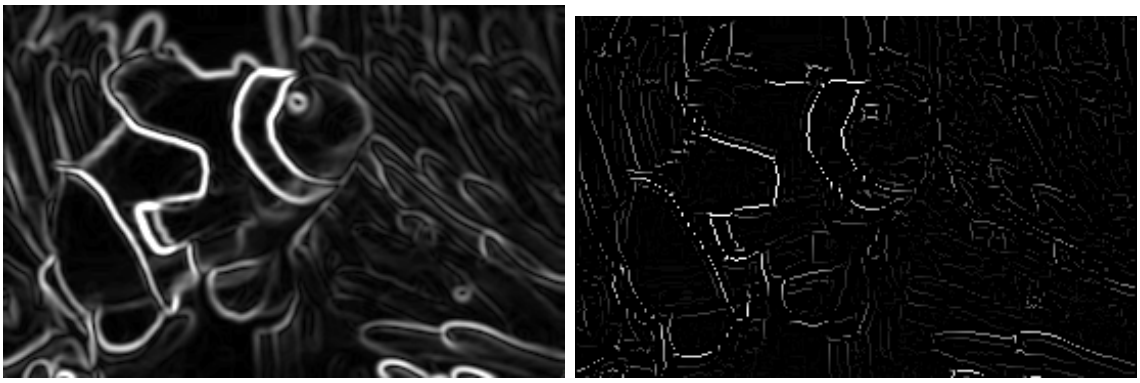


Figura 4: Comparación entre bordes de Sobel (izquierda) y Canny (derecha)

3.6.1. Constante de Estandarización para Canny

De igual manera que hemos comentado antes con el filtro de Sobel, ahora también aplicamos la constante de estandarización al filtro de Sobel utilizado en Canny, como una exploración adicional:

Código 3.7: Canny con filtro estandarizado

```
sobel_filter_standard = (1/8)*np.array([
                        [-1, 0, 1],
                        [-2, 0, 2],
                        [-1, 0, 1]
                    ], dtype=np.float32)

canny_edges_standard = [canny_edge_detector(
                            img=img,
                            sobel_filter=sobel_filter_standard,
                            gauss_sigma=gauss_sigma,
                            gauss_filter_shape=gauss_filter_shape,
                            verbose=True)
    for img in imgs]
```

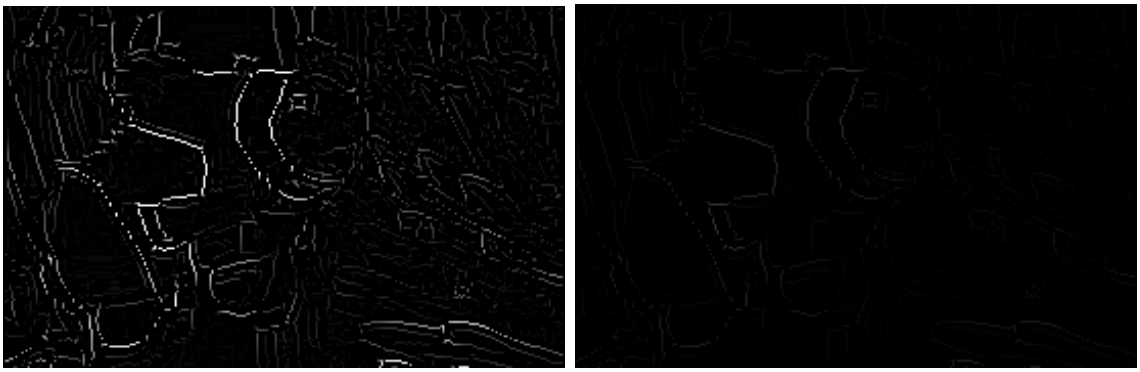


Figura 5: Imagen con canny (izquierda) y canny estandarizado (derecha)

4. Preguntas Adicionales de la Parte A

4.1. Pregunta A.1: Efecto del Ruido en la Detección de Bordes

Investigamos cómo afecta el ruido a la detección de bordes para así comprender mejor la importancia del filtrado gaussiano. Para ello, primero implementamos una función que nos permita añadir ruido gaussiano a nuestras imágenes:

Código 4.0: Función para añadir ruido gaussiano

```python
def add_gaussian_noise(img, mean=0, sigma=25):
    noisy = img + np.random.normal(mean, sigma, img.shape)
    noisy = np.clip(noisy, 0, 255).astype(np.uint8)
    return noisy

# Creación de imágenes con ruido
noisy_images = [add_gaussian_noise(img=img, sigma=16) for img in imgs]
```



Figura 6: Imagen original (izquierda) y con ruido gaussiano añadido (derecha)

4.1.1. Comparación con y sin Filtro Gaussiano

Aplicamos el detector de Sobel a las imágenes ruidosas, comparando los resultados que se obtienen cuando se aplica o no el filtro gaussiano:

Código 4.1: Comparación con imágenes ruidosas

```
sigma_with_gaussian_filter = 1
sigma_without_gaussian_filter = 0

sobel_filter = np.array([
                    [-1, 0, 1],
                    [-2, 0, 2],
                    [-1, 0, 1]
                        ], dtype=np.float32)

# Bordes con filtro gaussiano
sobel_edges_noisy_imgs_with_gaussian_filter = [

    sobel_edge_detector(
            img=img,
            filter=sobel_filter,
            gauss_sigma=sigma_with_gaussian_filter,
            gauss_filter_shape=None,
            verbose=False)[0]

for img in noisy_images]

# Bordes sin filtro gaussiano
sobel_edges_noisy_imgs_without_gaussian_filter = [

    sobel_edge_detector(
            img=img,
            filter=sobel_filter,
            gauss_sigma=sigma_without_gaussian_filter,
            gauss_filter_shape=None,
            verbose=False)[0]

for img in noisy_images]
```

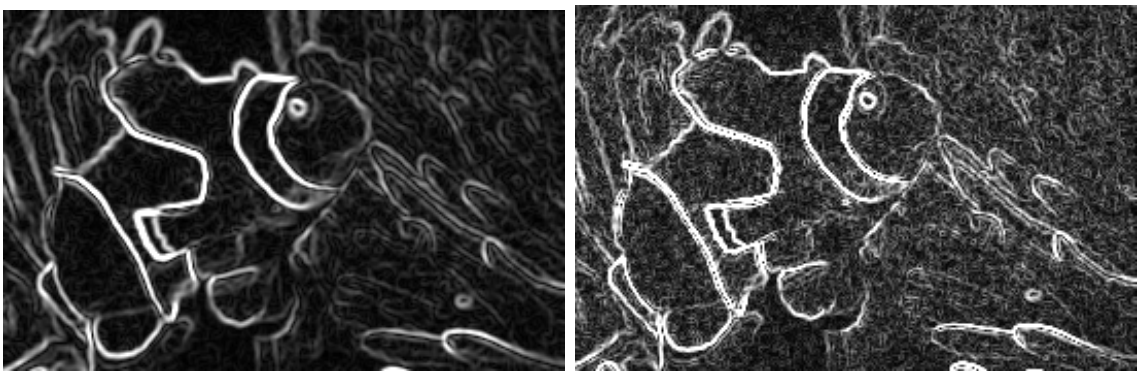


Figura 7: Bordes en imagen ruidosa con filtro gaussiano (izquierda) y sin filtro (derecha)

4.1.2. Análisis de Resultados

La comparación muestra claramente que:

- **Sin filtro gaussiano**: Se detectan numerosos bordes falsos producidos por el ruido, incluyendo bordes "secundarios"no deseados.
- **Con filtro gaussiano**: Se elimina la mayoría del ruido, preservando solo los bordes principales de la figura.
- El filtro gaussiano es esencial para obtener detecciones de bordes limpias y significativas en imágenes con ruido.

4.1.3. Comparaciones adicionales

Adicionalmente, hemos querido comprobar cuáles son los resultados de aplicar los detectores de Canny y Sobel a distintas imágenes las cuáles tienen más o menos ruido.

Para ello, hemos tratado de detectar los bordes de las imágenes, pero variando el parámetro sigma con el que les introducimos ruido a dichas imágenes. Hemos obtenido los siguientes resultados:

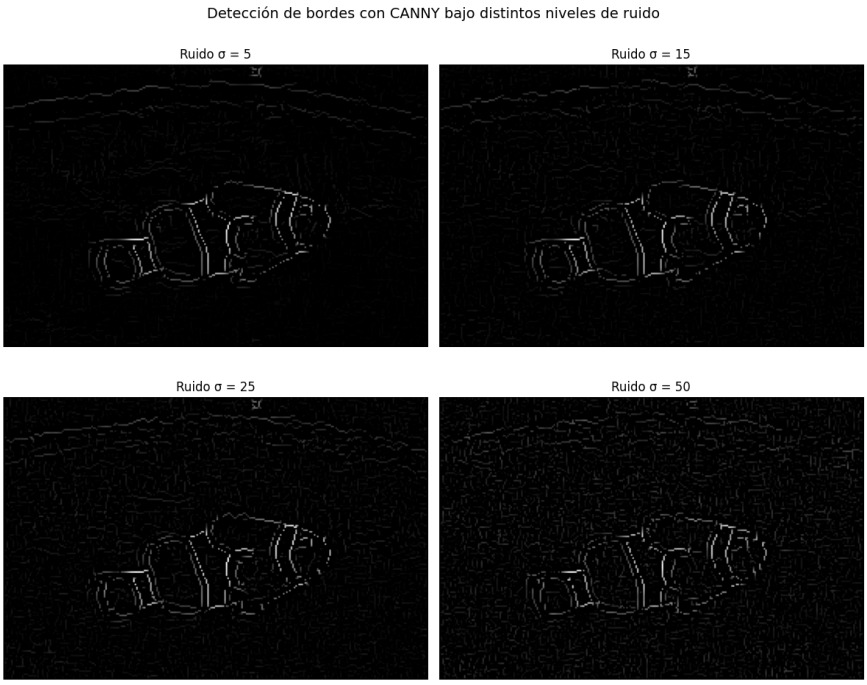


Figura 8: Detección de bordes con Canny variando el parámetro del ruido

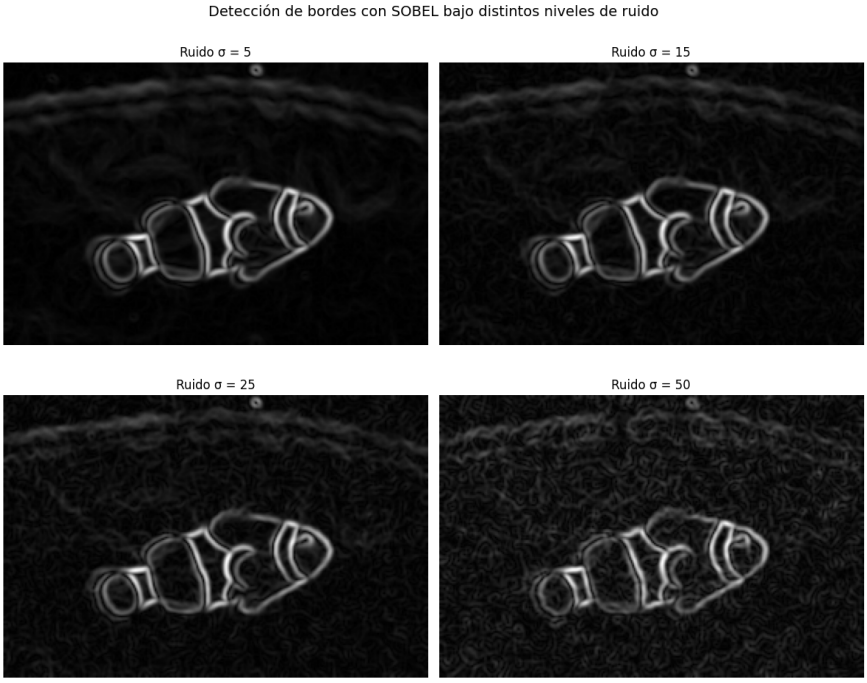


Figura 9: Detección de bordes con sobel cambiando el parámetro del ruido

4.2. Pregunta A.2: Comparación con scikit-image

Utilizamos la librería scikit-image para comparar los filtros Sobel, Canny y Prewitt:

Código 4.2: Comparación con scikit-image

```
from skimage import filters, feature
import skimage

# Aplicación de diferentes detectores de scikit-image
sobel_skimage = [

    filters.sobel(
        cv2.cvtColor(img, cv2.COLOR_BGR2GRAY))

for img in imgs]

canny_skimage = [

    feature.canny(
        cv2.cvtColor(img, cv2.COLOR_BGR2GRAY),
        sigma=1.4)
for img in imgs]

prewitt_skimage = [

    filters.prewitt(
    cv2.cvtColor(img, cv2.COLOR_BGR2GRAY))

for img in imgs]
```

4.2.1. Análisis Comparativo

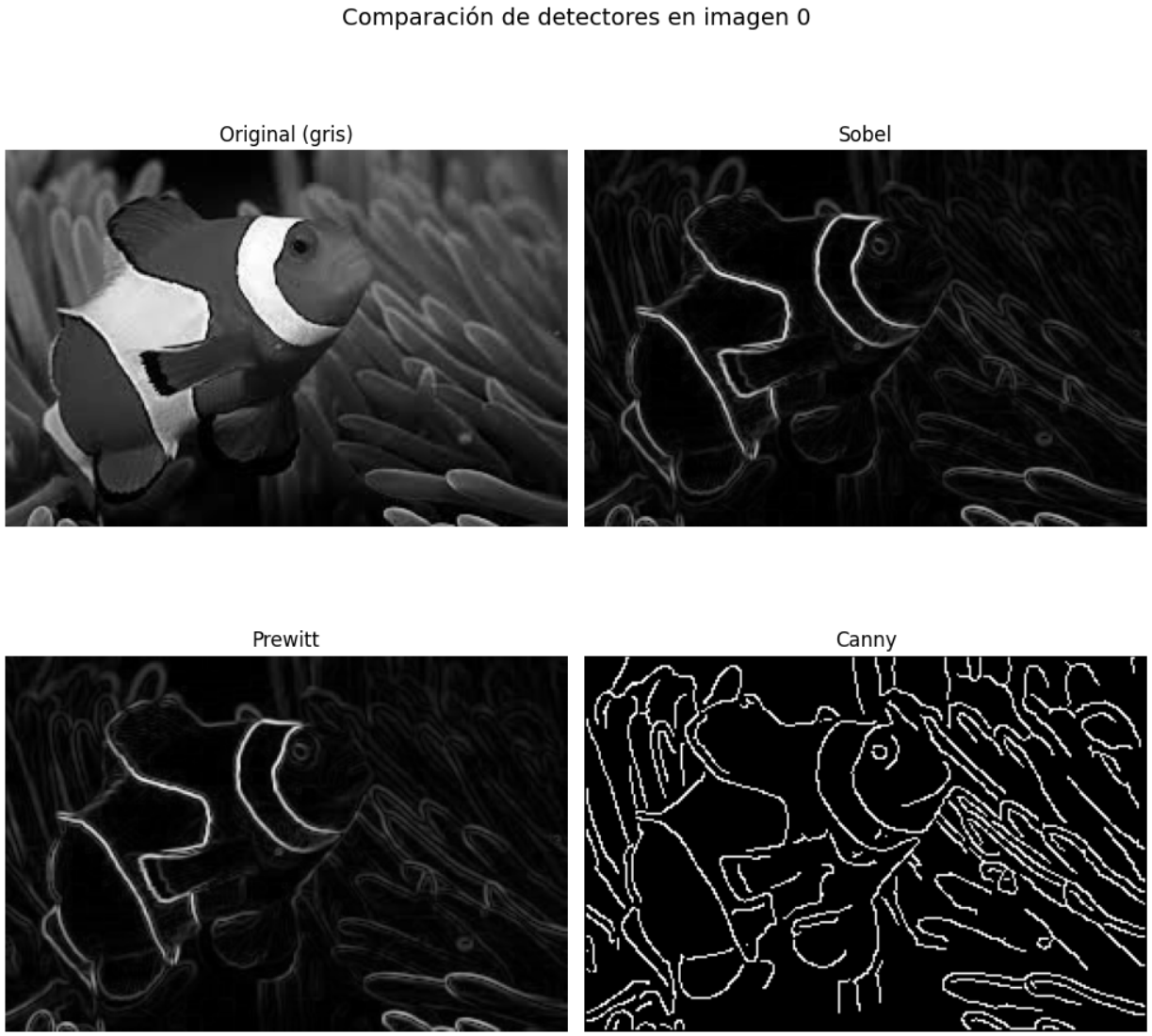


Figura 10: Comparación de detectores de scikit-image en la imagen 0: Sobel (arriba derecha), Canny (abajo derecha), Prewitt (abajo izquierda)

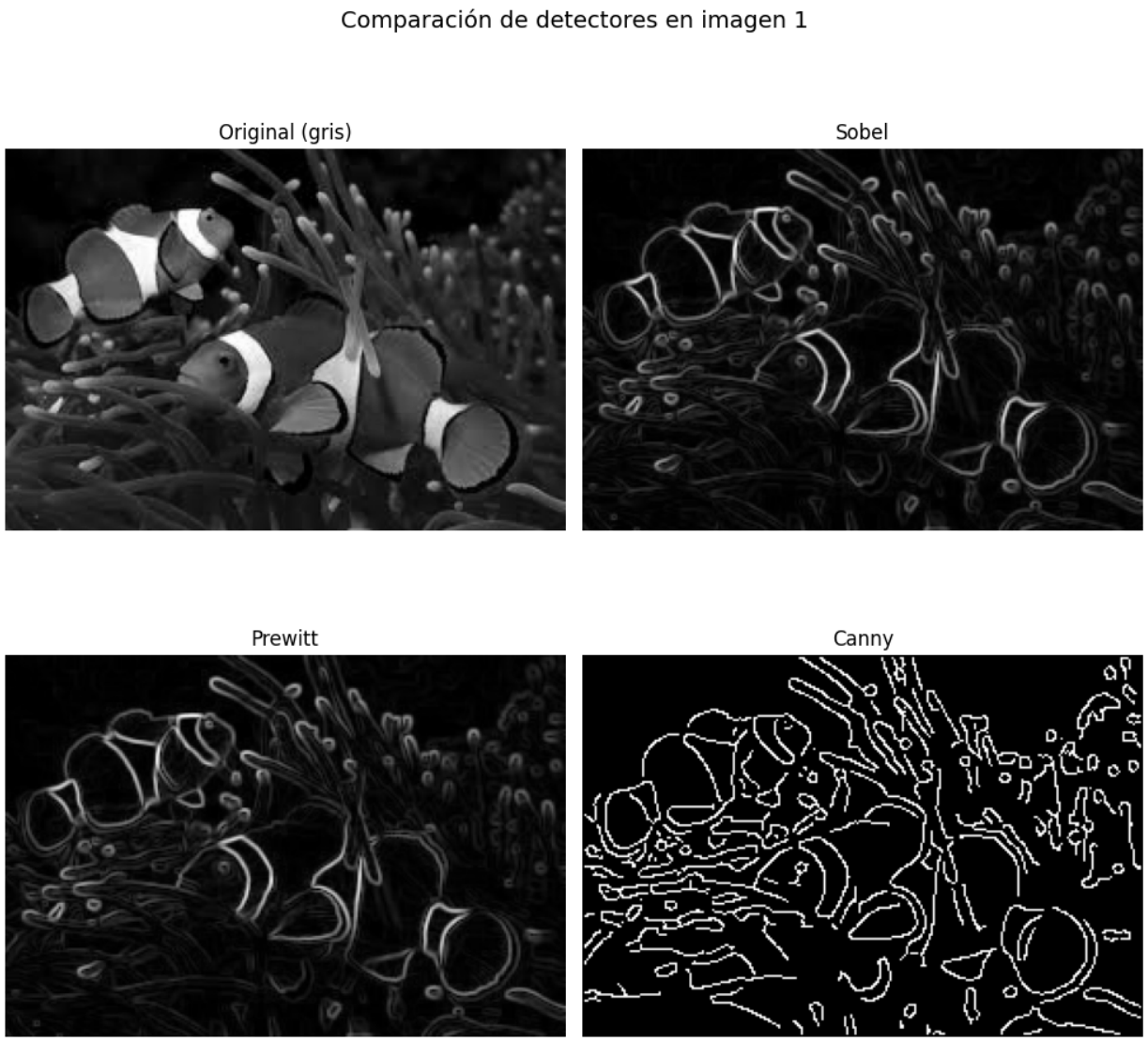


Figura 11: Comparación de detectores de scikit-image en la imagen 1: Sobel (arriba derecha), Canny (abajo derecha), Prewitt (abajo izquierda)

Las principales diferencias observadas son:

- **Sobel**: Proporciona bordes gruesos y es más robusto al ruido que Prewitt.
- **Canny**: Produce bordes delgados, continuos y bien conectados, con mejor supresión de falsos positivos, siendo el método más preciso y selectivo, pero también el que conlleva mayor coste computacional.
- **Prewitt**: Similar a Sobel pero es más sensible al ruido y los bordes que detecta aparecen menos contrastados.
- **Patrón común**: Todos los detectores identifican los mismos bordes estructurales principales, difiriendo principalmente en el grosor y la continuidad de los bordes detectados.

En definitiva, Prewitt y Sobel son detectores rápidos y sencillos, adecuados para un realce básico de contornos, mientras que Canny ofrece resultados más limpios y detallados, apropiados cuando se requiere una detección de bordes de alta calidad.

5. Parte B: Operadores Morfológicos

En esta parte de la práctica nos centramos en el estudio e implementación de operadores morfológicos fundamentales para el procesamiento de imágenes binarias. Los operadores morfológicos son técnicas no lineales que permiten modificar la forma y estructura de los objetos en una imagen, siendo especialmente útiles en tareas de segmentación, eliminación de ruido y extracción de características.

5.1. Tarea B.1: Implementación del Método de Binarización

El primer paso fundamental para aplicar operadores morfológicos es convertir las imágenes a formato binario. Implementamos dos versiones del método `binarize`:

Código 5.0: Implementación del método de binarización

```python
def binarize(img: np.array, threshold: int = 127):
    # Convertimos a escala de grises si la imagen es RGB
    if len(img.shape)== 3:
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    else:
        gray = img

    # Aplicamos umbralización manualmente usando numpy
    binary_img = np.where(gray < threshold, 0, 255).astype(np.uint8)
    # Todo lo que es menor que el threshold, lo ponemos como 0, y el resto
    ↪  como 1

    return binary_img
```

Adicionalmente, implementamos una versión alternativa utilizando la función de threshold de OpenCV:

Código 5.1: Binarización usando OpenCV

```python
def binarize2(img: np.array, threshold: int = 127):
    # Convertimos a escala de grises si la imagen es RGB
    if len(img.shape)== 3:
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    else:
```

```
        gray = img

    # Aplicamos umbralización usando threshold
    _, binary_img = cv2.threshold(gray, threshold, 255, cv2.THRESH_BINARY)
    # Todo lo que es menor que el threshold, se pone como 0, y el resto como
    ↪  1

    return binary_img
```

5.1.1. Explicación del Proceso de Binarización

El proceso de binarización implementado sigue estos pasos:

1. **Conversión a escala de grises**: Si la imagen es RGB, se convierte a escala de grises para trabajar con un solo canal de intensidad.
2. **Aplicación de umbral**: Se establece un valor de umbral (por defecto 127) para clasificar los píxeles.
3. **Asignación binaria**: Los píxeles con valor menor al umbral se establecen a 0 (negro) y los mayores o iguales a 255 (blanco).

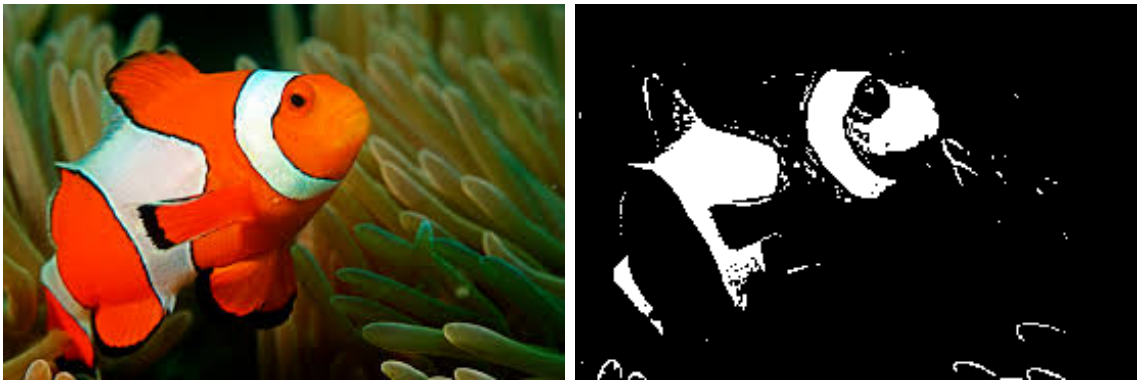


Figura 12: Imagen original (izquierda) y resultado de la binarización (derecha)

5.2. Tarea B.2: Implementación del Método de Dilatación

La dilatación es un operador morfológico que expande las regiones brillantes de una imagen. Implementamos la función `custom_dilate`:

Código 5.2: Implementación del método de dilatación

```
# Definimos el tamaño de nuestro kernel, que actuará como elemento
↪ estructurante
kernel_shape = (3,3)
structurant_element = np.ones(shape=kernel_shape).astype(np.uint8)

def custom_dilate(img):
    # Binarizamos la imagen antes de nada (por si acaso no lo está)
    img = binarize(img)

    # Padding de la imagen original para mantener dimensiones después del
    ↪ procesamiento
    pad_width = int((kernel_shape[0] - 1) / 2)  # investigando en internet
    ↪ sobre cómo se relaciona el tamaño del kernel aplicado y el padding
    ↪ necesario, hallamos esta fórmula

    padded = np.pad(array=img, pad_width=pad_width, mode="mean")  # hacemos
    ↪ padding usando un kernel de (3x3) por lo que tenemos que añadir solo
    ↪ 1 fila y columna tanto arriba como abajo de la imagen
    # si usamos el modo "mean" significa que el valor que se añade de
    ↪ padding será la media de los valores cercanos en ese pixel de la
    ↪ imagen

    # Obtenemos dimensiones de la imagen
    width = img.shape[1]
    height = img.shape[0]

    # Creamos un elemento con las mismas dimensiones que la imagen con
    ↪ padding
    dilated = np.zeros(shape=padded.shape)

    # Recorremos la imagen aplicando la operación de dilatación
    for j in range(pad_width, pad_width + height):
        for i in range(pad_width, pad_width + width):
            # Seleccionamos el área involucrada según nuestro elemento
            ↪ estructurante
            area_involved = padded[j - pad_width:j + pad_width + 1,
                                   i - pad_width:i + pad_width +
                                   ↪ 1]*structurant_element
            # cogemos una matriz de tamaño kernel_shape, centrada en nuestro
            ↪ elemento j,i y luego lo multiplicamos por el elemento
            ↪ estructurante

            # Cogemos el máximo de todos los valores de nuestro área
            ↪ seleccionada
            dilated[j, i] = np.max(area_involved)
```

```
    # Seleccionamos la región de interés (ROI)
    dilated = dilated[pad_width:height+1, pad_width:width+1]  # a la hora de
    ↪  devolver la imagen, nos quedamos con lo que no es parte añadida por
    ↪  el padding

    return dilated.astype(np.uint8)  # devolvemos la imagen convertida a una
    ↪  matriz de números enteros
```



Figura 13: Imagen 0 binarizada (izquierda) y aplicado el método de dilatación (derecha)

5.2.1. Explicación del Proceso de Dilatación

La dilatación implementada sigue este procedimiento:

1. **Preparación de la imagen**: Se binariza la imagen y se aplica padding para manejar los bordes.
2. **Elemento estructurante**: Se define un kernel de 3x3 píxeles que actúa como elemento estructurante.
3. **Procesamiento por ventanas**: Para cada píxel, se examina su vecindario definido por el elemento estructurante.
4. **Operación de máximo**: Se asigna al píxel central el valor máximo encontrado en su vecindario.
5. **Extracción de ROI**: Se elimina el padding añadido para obtener la imagen final.

5.3. Tarea B.3: Implementación del Método de Erosión

La erosión es el operador morfológico dual a la dilatación, que contrae las regiones brillantes de una imagen. Implementamos la función `custom_erode`:

Código 5.3: Implementación del método de erosión

```
# Definimos el tamaño de nuestro kernel, elemento estructurante
kernel_shape = (3,3)
structurant_element = np.ones(shape=kernel_shape).astype(np.uint8)

def custom_erode(img):
    # Binarizamos la imagen antes de nada (por si acaso no lo está)
    img = binarize(img)

    # Padding de la imagen original para mantener dimensiones
    pad_width = int((kernel_shape[0] - 1) / 2)

    padded = np.pad(array=img, pad_width=pad_width, mode="mean")
    # hacemos padding usando un kernel de (3x3) por lo que tenemos  que
    ↪  añadir solo 1 fila y columna tanto arriba como abajo de la imagen

    # Obtenemos dimensiones de la imagen
    width = img.shape[1]
    height = img.shape[0]

    # Mismas dimensiones que la imagen con padding
    eroded = np.zeros(shape=padded.shape)

    # Recorremos la imagen aplicando la operación de erosión
    for j in range(pad_width, pad_width + height):
        for i in range(pad_width, pad_width + width):
            # Seleccionamos el área involucrada
            area_involved = padded[j - pad_width:j + pad_width + 1,
                                   i - pad_width:i + pad_width +
                                   ↪  1]*structurant_element
            # cogemos una matriz de tamaño kernel_shape, centrada en nuestro
            ↪  elemento j,i y multiplicamos por elemento estructurante

            # Cogemos el mínimo de todos los valores
            eroded[j, i] = np.min(area_involved)

    # Seleccionamos la región de interés (ROI)
    eroded = eroded[pad_width:height+1, pad_width:width+1]
    # Nos quedamos con lo que no es parte añadida por el padding

    return eroded.astype(np.uint8)  # imagen convertida enteros
```



Figura 14: Imagen 0 binarizada (izquierda) y aplicado el método de erosión (derecha)

5.3.1. Explicación del Proceso de Erosión

La erosión implementada sigue un procedimiento similar a la dilatación pero con diferencias clave:

1. **Preparación idéntica**: Mismo proceso de binarización y padding.
2. **Mismo elemento estructurante**: Kernel de 3x3 píxeles.
3. **Operación de mínimo**: En lugar del máximo, se asigna al píxel central el valor mínimo encontrado en su vecindario.
4. **Extracción de ROI**: Se elimina el padding añadido para obtener la imagen final.
5. **Efecto contrario**: Mientras la dilatación expande las regiones blancas, la erosión las contrae.

5.4. Pregunta B.1: Aplicación de los Operadores Morfológicos

Aplicamos ambos operadores morfológicos a todas las imágenes de la carpeta `data`:

Código 5.4: Aplicación de operadores morfológicos

```
# Imágenes dilatadas
dilated_imgs = [custom_dilate(img) for img in imgs]

# Imágenes erosionadas
eroded_imgs = [custom_erode(img) for img in imgs]
```

5.4.1. Análisis de Resultados

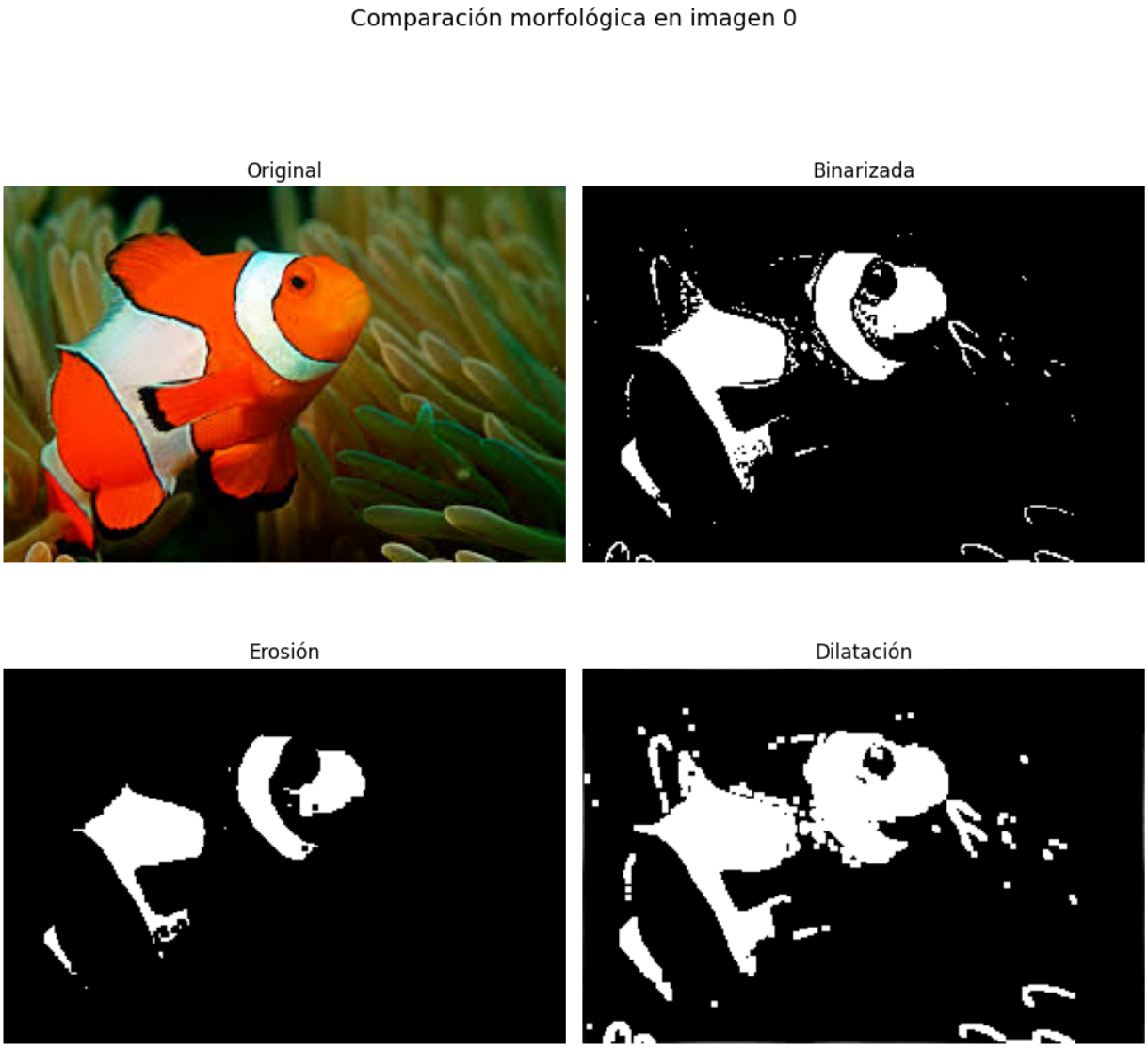


Figura 15: Comparación de operadores morfológicos en la imagen 0

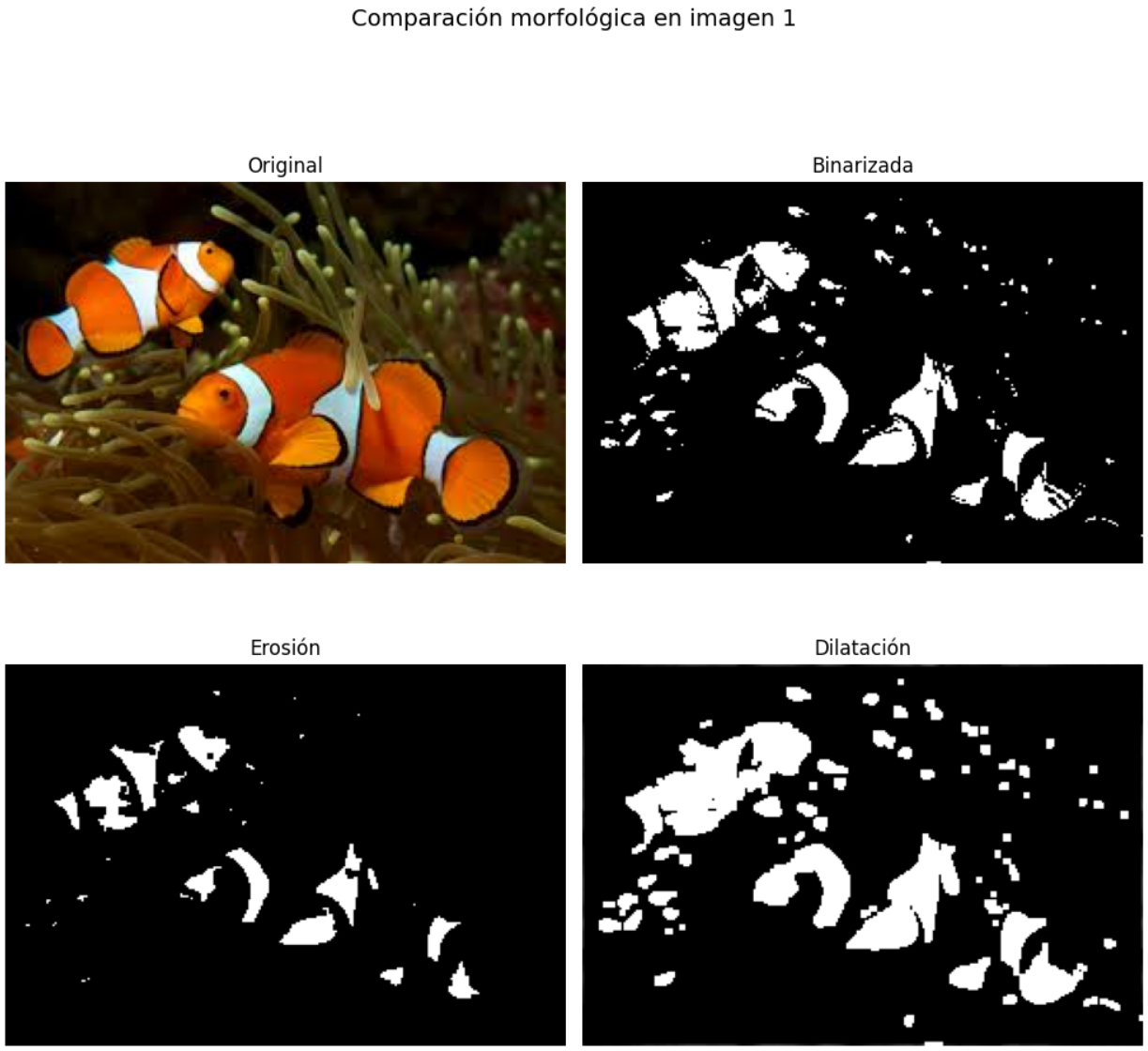


Figura 16: Comparación de operadores morfológicos en la imagen 1

Los resultados muestran claramente los efectos de cada operador:

- **Dilatación**: Expande las regiones blancas, rellenando pequeños agujeros y conectando regiones cercanas.
- **Erosión**: Contrae las regiones blancas, eliminando pequeños detalles y separando regiones conectadas.
- **Complementariedad**: Ambos operadores son duales y pueden combinarse para operaciones más complejas como apertura y cierre.

5.4.2. Aplicaciones Prácticas de los Operadores Morfológicos

Los operadores implementados tienen numerosas aplicaciones en procesamiento de imágenes:

- **Limpieza de imágenes**: Eliminación de ruido y pequeños artefactos.
- **Segmentación**: Separación de objetos conectados y relleno de huecos.
- **Extracción de características**: Aislamiento de formas y estructuras específicas.
- **Preprocesamiento**: Preparación de imágenes para operaciones posteriores como OCR o análisis de formas.

La implementación manual de estos operadores nos ha permitido comprender en profundidad su funcionamiento interno y los efectos que producen sobre diferentes tipos de imágenes binarias.

6. Parte C: Detección de Esquinas

El objetivo de este apartado es detectar las esquinas presentes en las imágenes de la carpeta `data/partC/source`. Las esquinas son puntos de interés que representan intersecciones de bordes y son especialmente útiles para aplicaciones como registro de imágenes, seguimiento de objetos y reconstrucción 3D.

6.1. Tarea C.1: Creación de Carpeta para Resultados

Comenzamos creando una carpeta específica para almacenar los resultados de esta parte de la práctica:

Código 6.0: Creación de carpeta para resultados

```
# Crear carpeta para guardar todos los resultados de la parte C
folder_name = "partC_processed"
folder_path = os.path.join("..", "data", "partC", "source", folder_name)
os.makedirs(name=folder_path, exist_ok=True)
```

6.2. Tarea C.2: Métodos de Carga de Imágenes

Implementamos y comparamos dos métodos diferentes para cargar imágenes, destacando las diferencias en el manejo de los canales de color:

Código 6.1: Métodos de carga de imágenes

```
def imageio_load_images(filenames: List) -> List:
    '''
    Cargar imágenes usando imageio.imread (formato RGB)
    '''
    return [imageio.v2.imread(filename) for filename in filenames]

def opencv_load_images(filenames: List) -> List:
    '''
    Cargar imágenes usando cv2.imread (formato BGR)
    '''
    return [cv2.imread(filename) for filename in filenames]

# Crear listas ordenadas con los paths de todas las imágenes
imgs_C_path_png = '../data/partC/*.png'
imgs_C_path_jpg = '../data/partC/*.jpg'
source_paths = sorted(glob.glob(imgs_C_path_png) +
↪  glob.glob(imgs_C_path_jpg))

imageio_images = imageio_load_images(source_paths)
opencv_images = opencv_load_images(source_paths)

# Guardar ambas imágenes usando cv2.imwrite()
cv2.imwrite(os.path.join(folder_path, "tennis_court_imageio.jpg"),
↪  imageio_images[-1])
cv2.imwrite(os.path.join(folder_path, "tennis_court_opencv.jpg"),
↪  opencv_images[-1])
```

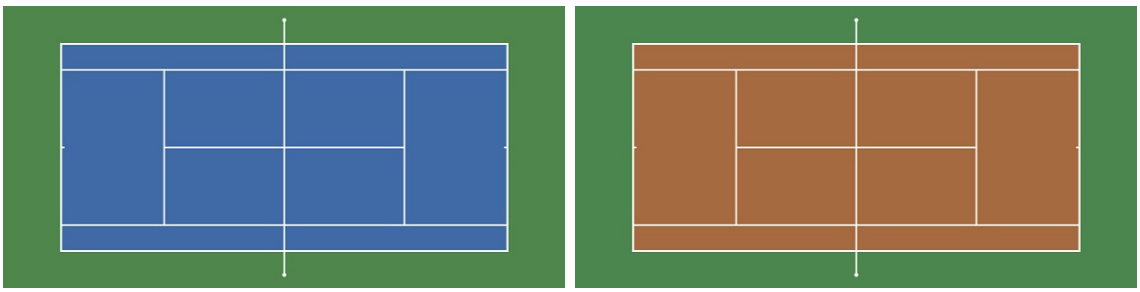


Figura 17: Resultados carga de imagen con opencv (izquierda) y con imageio (derecha)

6.2.1. Análisis de Diferencia entre Métodos de Carga

Al guardar ambas imágenes con la misma función `cv2.imwrite()`, se observa que:

- La imagen cargada con `opencv.imread()` mantiene sus colores originales (azul y verde)

- La imagen cargada con `imageio.v2.imread()` aparece con los colores alterados (donde había azul ahora se ve rojo)

Esto ocurre porque cada librería utiliza un orden distinto de canales de color:

- **OpenCV (cv2)** carga las imágenes en formato **BGR** (Blue, Green, Red)
- **ImageIO** carga las imágenes en formato **RGB** (Red, Green, Blue)

Cuando ambas imágenes se guardan con `cv2.imwrite()`, esta función no realiza conversión de color, sino que interpreta directamente los valores numéricos. Por tanto, si le pasamos una imagen en formato RGB (como la de imageio), OpenCV la trata erróneamente como si fuera BGR, intercambiando los canales rojo y azul.

6.3. Tarea C.3: Implementación del Detector de Esquinas de Harris

El detector de esquinas de Harris se basa en analizar cómo varía la intensidad de una pequeña ventana de píxeles al desplazarse en distintas direcciones dentro de la imagen.

Su principio fundamental es que una esquina es un punto en el que la intensidad cambia significativamente en todas las direcciones, a diferencia de una región plana (sin cambios) o un borde (cambio solo en una dirección).

Implementamos la función `harris_corner_detector`:

Código 6.2: Implementación del detector de Harris

```
def harris_corner_detector(image: np.array, blockSize: int, ksize: int, k:
↪  float):

    # La imagen de entrada al detector de Harris debe estar en escala de
    ↪  grises
    gray = cv2.cvtColor(src=image, code=cv2.COLOR_BGR2GRAY)

    # La imagen de entrada debe ser de tipo float32
    gray = gray.astype(np.float32)

    # Aplicar detección de esquinas de Harris
    harris = cv2.cornerHarris(src=gray, blockSize=blockSize, ksize=ksize,
    ↪  k=k)

    # El resultado se dilata para marcar las esquinas (no es importante)
    harris = cv2.dilate(src=harris, kernel=None)
```

```
    # Umbral para un valor óptimo del 1% del valor R máximo
    # Si el valor del píxel > 1% del valor máximo, se debe resaltar como
    ↪  esquina roja
    max_val = np.max(harris)  # Obtenemos el valor máximo
    th = 0.01*max_val  # Definimos nuestro umbral usando 0.01

    for fila in range(harris.shape[0]):
        for columna in range(harris.shape[1]):
            if harris[fila][columna] > th:  # Comparamos todos los píxeles
            ↪  con el valor del umbral
                image[fila][columna] = [0, 0, 255]  # En caso de cumplirse
                ↪  la condición, ponemos rojo máximo (en BGR)

    return image
```

6.4. Tarea C.4: Aplicación del Detector de Harris

Aplicamos el detector de Harris a las imágenes de trabajo:

Código 6.3: Aplicación del detector de Harris

```
# Crear carpeta donde guardar las imágenes
folder_name = "partC-D-E"
folder_path = os.path.join("..", "data", folder_name)
os.makedirs(name=folder_path, exist_ok=True)

# Definir parámetros para la función de detección de esquinas de Harris
blockSize = 2
ksize = 3
k = 0.04

# Primera imagen - geometry
save_name = "geometry"
image = opencv_images[1].copy()
harris_image = harris_corner_detector(image, blockSize=blockSize,
↪  ksize=ksize, k=k)
cv2.imwrite(os.path.join(folder_path, f"Harris_{save_name}.jpg"),
↪  harris_image)

# Segunda imagen - football
save_name = "football"
image = opencv_images[0].copy()
harris_image = harris_corner_detector(image, blockSize=blockSize,
↪  ksize=ksize, k=k)
```

```
cv2.imwrite(os.path.join(folder_path, f"Harris_{save_name}.jpg"),
↪  harris_image)
```

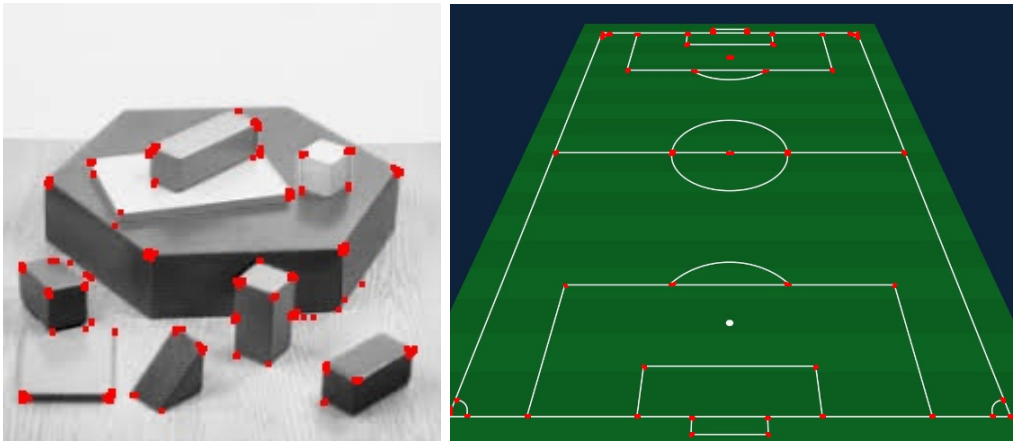


Figura 18: Resultados del detector de Harris: geometría (izquierda) y fútbol (derecha)

6.5. Tarea C.5: Implementación del Detector de Esquinas de Shi-Tomasi

El detector de Shi-Tomasi es una mejora del detector de Harris que utiliza el valor mínimo de los autovalores de la matriz de estructura. Implementamos la función `shi_tomasi_corner_detection`:

Código 6.4: Implementación del detector de Shi-Tomasi

```
def shi_tomasi_corner_detection(
                    image: np.array,
                    maxCorners: int,
                    qualityLevel:float,
                    minDistance: int,
                    corner_color: tuple,
                    radius: int):

    # La imagen de entrada al detector de Tomasi debe estar en escala de
    ↪  grises
    gray = cv2.cvtColor(src=image, code=cv2.COLOR_BGR2GRAY)

    # Aplicar función cv2.goodFeaturesToTrack
    corners = cv2.goodFeaturesToTrack(
        image=gray,
        maxCorners=maxCorners,
        qualityLevel=qualityLevel,
        minDistance=minDistance)
```

```
    # Conversión de coordenadas de esquinas a enteros
    corners = np.intp(corners)

    for corner in corners:  # Iteramos por cada una de las esquinas
    ↪  encontradas
        # Array multidimensional a array aplanado, si es necesario
        x, y = corner.ravel()  # Obtenemos las coordenadas de cada esquina

        # Dibujar círculos en las esquinas
        cv2.circle(image, center=(x,y), radius=radius, color=corner_color)

    return image
```

6.6. Tarea C.6: Aplicación del Detector de Shi-Tomasi

Aplicamos el detector de Shi-Tomasi a las imágenes de trabajo:

Código 6.5: Aplicación del detector de Shi-Tomasi

```
# Crear carpeta donde guardar las imágenes (ya debería estar creada)
folder_name = "partC-D-E"
folder_path = os.path.join("..", "data", folder_name)
os.makedirs(name=folder_path, exist_ok=True)

# Definir parámetros para la función de detección de bordes de Harris
maxCorners = 50
qualityLevel = 0.05  # No muy bajo para no detectar esquinas de mala calidad
minDistance = 10
radius = 4

# Primera imagen - Esquinas moradas y radio = 4
save_name = "geometry"
purple_color = (128, 0, 128)
image = opencv_images[1].copy()
tomasi_image = shi_tomasi_corner_detection(image, maxCorners=maxCorners,
                                           qualityLevel=qualityLevel,
                                           minDistance=minDistance,
                                           corner_color=purple_color,
                                           ↪  radius=radius)

cv2.imwrite(os.path.join(folder_path, f"Shi-Tomasi_{save_name}.jpg"),
↪  tomasi_image)
```

```
# Segunda imagen - Esquinas naranjas y radio = 4
save_name = "football"
orange_color = (0, 165, 255)  # En formato BGR
image = opencv_images[0].copy()
tomasi_image = shi_tomasi_corner_detection(image, maxCorners=maxCorners,
                                           qualityLevel=qualityLevel,
                                           minDistance=minDistance,
                                           corner_color=orange_color,
                                           ↪  radius=radius)

cv2.imwrite(os.path.join(folder_path, f"Shi-Tomasi_{save_name}.jpg"),
↪  tomasi_image)
```

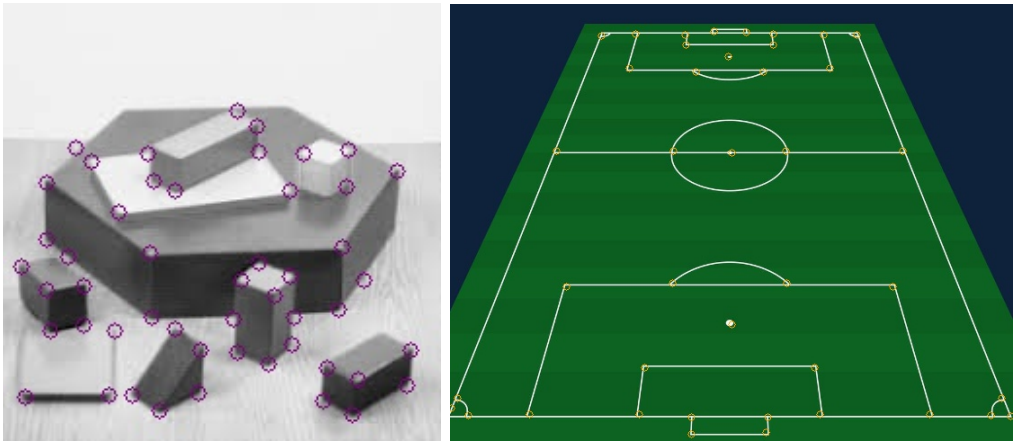


Figura 19: Resultados del detector de Shi-Tomasi: geometría (izquierda) y fútbol (derecha)

6.7. Pregunta C.1: Detección de Esquinas con Harris en Sudoku y Tennis

Se aplica el detector de Harris a las dos imágenes restantes de la carpeta `data/partC-D-E/source`: sudoku y tennis.

6.7.1. Definición de Parámetros

Se definen los parámetros para la función de detección de esquinas de Harris, basándose en valores estándar encontrados en la documentación y ejemplos:

Código 6.6: Parámetros para el detector de Harris

```
# Parámetros para la detección de esquinas de Harris
blockSize = 2  # Tamaño del vecindario considerado para la detección
ksize = 3      # Tamaño de apertura para el operador Sobel
k = 0.04       # Parámetro libre del detector de Harris
```

6.7.2. Aplicación a las Imágenes

Código 6.7: Detección de Harris en sudoku y tennis

```
# Primera imagen - Sudoku
save_name = "sudoku"

# Copiar imagen original
image = opencv_images[2].copy()

# Aplicar detección de esquinas de Harris
harris_image = harris_corner_detector(image, blockSize=blockSize,
                                      ksize=ksize, k=k)

# Guardar imagen resultante
cv2.imwrite(os.path.join(folder_path, f"Harris_{save_name}.jpg"),
↪  harris_image)

# Segunda imagen - Tennis
save_name = "tennis"

# Copiar imagen original
image = opencv_images[3].copy()

# Aplicar detección de esquinas de Harris
harris_image = harris_corner_detector(image, blockSize=blockSize,
                                      ksize=ksize, k=k)

# Guardar imagen resultante
cv2.imwrite(os.path.join(folder_path, f"Harris_{save_name}.jpg"),
↪  harris_image)
```

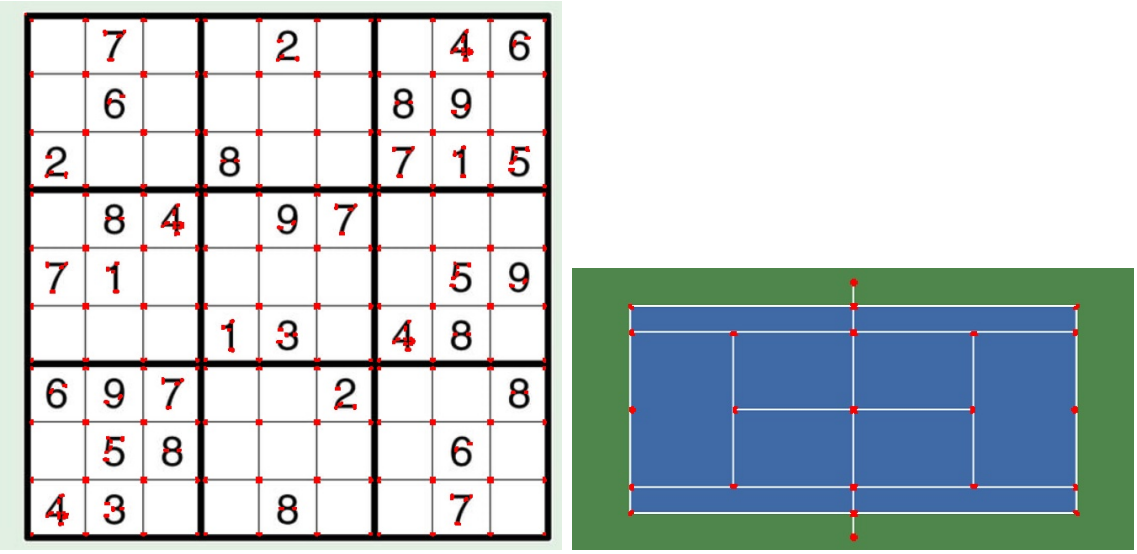


Figura 20: Resultados del detector de Harris: sudoku (izquierda) y tenis (derecha)

6.8. Pregunta C.2: Detección de Esquinas con Shi-Tomasi en Sudoku y Tennis

Se aplica el detector de Shi-Tomasi a las mismas imágenes (sudoku y tennis) para comparar los resultados con el método de Harris.

6.8.1. Definición de Parámetros

Se definen los parámetros para el detector de Shi-Tomasi:

Código 6.8: Parámetros para el detector de Shi-Tomasi

```
# Parámetros para la detección de esquinas de Shi-Tomasi
maxCorners = 100       # Número máximo de esquinas a detectar
qualityLevel = 0.05    # No muy bajo para evitar esquinas de mala calidad
minDistance = 10       # Distancia mínima euclidiana entre esquinas
↪  detectadas
radius = 6             # Radio de los círculos para visualizar las esquinas
```

El parámetro `qualityLevel` se mantiene en 0.05 para asegurar que solo se detecten esquinas de buena calidad, evitando puntos débiles o ruidosos.

6.8.2. Aplicación a las Imágenes

Código 6.9: Detección de Shi-Tomasi en sudoku y tennis

```
# Primera imagen - Sudoku con esquinas naranjas
save_name = "sudoku"

# Color naranja
orange_color = (0, 165, 255)

# Copiar imagen original
image = opencv_images[2].copy()

# Aplicar detección de esquinas de Shi-Tomasi
tomasi_image = shi_tomasi_corner_detection(image, maxCorners=maxCorners,
                                           qualityLevel=qualityLevel,
                                           minDistance=minDistance,
                                           corner_color=orange_color,
                                           radius=radius)

# Guardar imagen resultante
cv2.imwrite(os.path.join(folder_path, f"Shi-Tomasi_{save_name}.jpg"),
↪  tomasi_image)

# Segunda imagen - Tennis con esquinas naranjas
save_name = "tennis"

# Color naranja
orange_color = (0, 165, 255)

# Copiar imagen original
image = opencv_images[3].copy()

# Aplicar detección de esquinas de Shi-Tomasi
tomasi_image = shi_tomasi_corner_detection(image, maxCorners=maxCorners,
                                           qualityLevel=qualityLevel,
                                           minDistance=minDistance,
                                           corner_color=orange_color,
                                           radius=radius)

# Guardar imagen resultante
cv2.imwrite(os.path.join(folder_path, f"Shi-Tomasi_{save_name}.jpg"),
↪  tomasi_image)
```

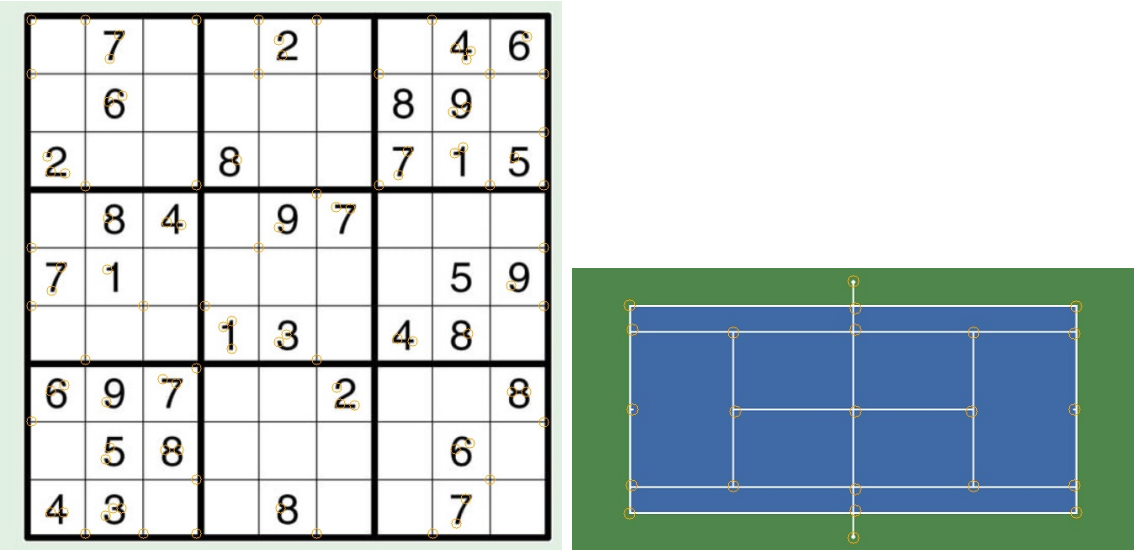


Figura 21: Resultados del detector de Shi-Tomasi: geometría (izquierda) y fútbol (derecha)

6.9. Comparación entre Harris y Shi-Tomasi

Realizamos una comparación visual entre ambos métodos con parámetros fijos:

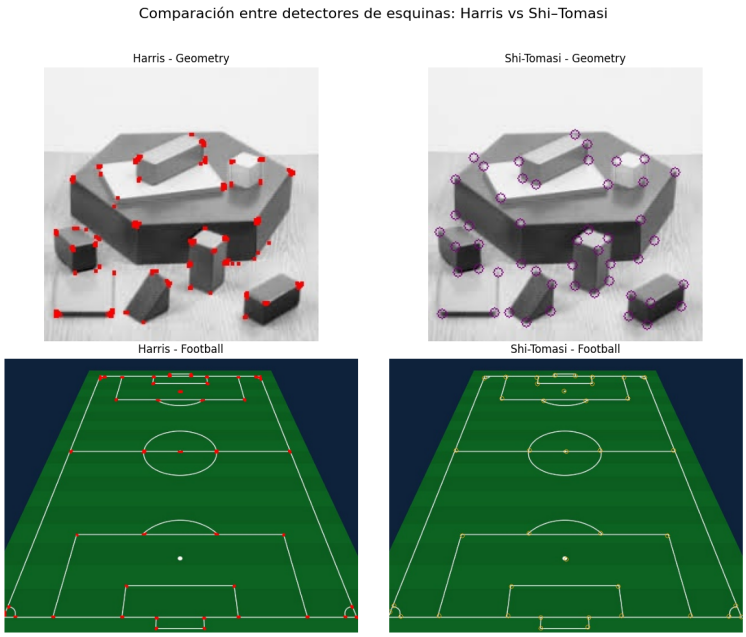


Figura 22: Comparación: Harris (izquierda) vs Shi-Tomasi (derecha)

También hemos decidido realizar unas comparaciones adicionales para el mismo detector de esquinas pero utilizando distintas combinaciones de parámetros:

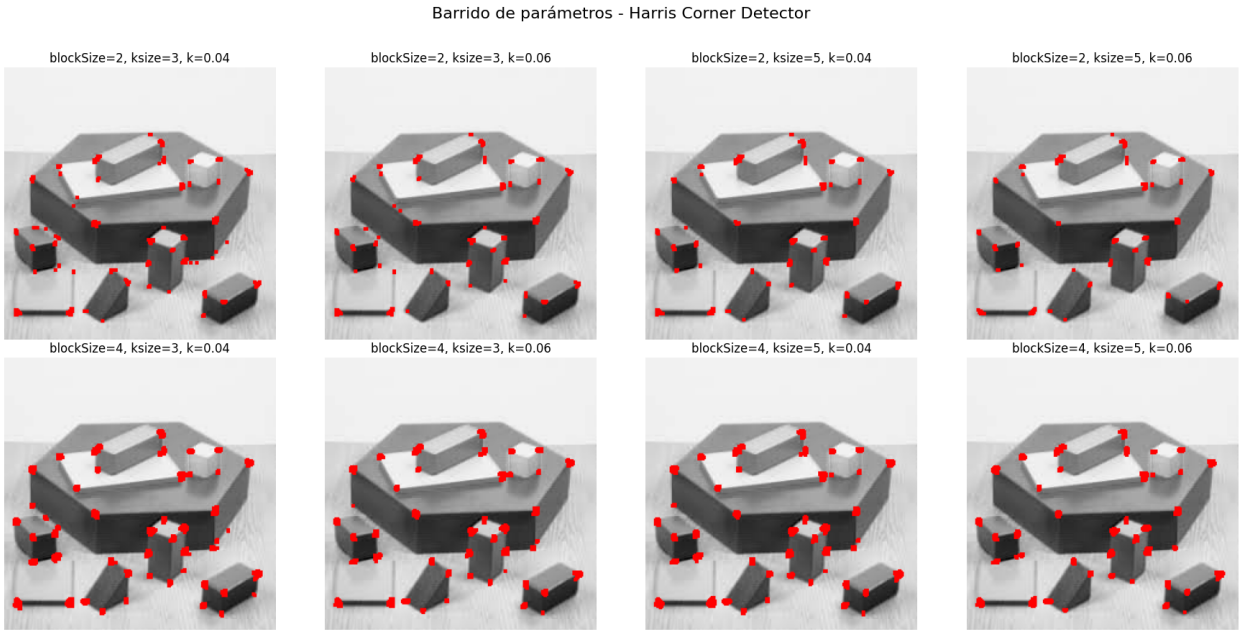


Figura 23: Comparación de Harris con distintos parámetros

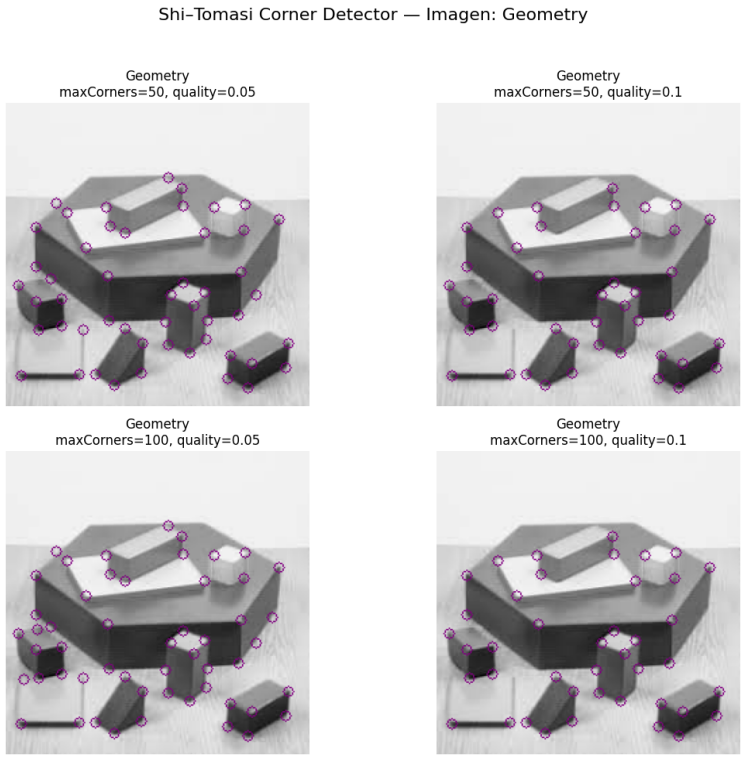


Figura 24: Comparación de Shi-Tomasi con distintos parámetros

7. Conclusión General

En esta práctica hemos implementado y comparado diferentes técnicas fundamentales en el procesamiento de imágenes, tanto para detección de bordes como de esquinas, así como el uso del filtro gaussiano para mejorar dicha detección en imágenes ruidosas:

7.1. Logros Principales

- **Filtrado Gaussiano**: Implementación exitosa del filtro gaussiano para suavizado de imágenes, demostrando su importancia en la eliminación de ruido antes de la detección de características.
- **Detección de Bordes**: Implementación completa de los operadores de Sobel y Canny, comprendiendo sus diferencias fundamentales en términos de precisión y robustez.
- **Detección de Esquinas**: Implementación y comparación de los detectores de Harris y Shi-Tomasi, analizando sus ventajas y desventajas en diferentes escenarios.
- **Análisis de Ruido**: Demostración clara de cómo el ruido afecta a la detección de características y la efectividad del filtrado gaussiano para mitigar este problema.
- **Comparación de Métodos**: Evaluación comparativa entre diferentes detectores.

7.2. Conclusiones Técnicas

- El **filtro gaussiano** es esencial como paso previo a la detección de características, especialmente en imágenes con ruido.
- El **operador de Sobel** es computacionalmente eficiente pero produce bordes gruesos y es sensible al ruido.
- El **algoritmo de Canny** proporciona bordes más finos y precisos gracias a la supresión de no máximos y la umbralización con histéresis. Aunque también es más costoso computacionalmente.
- El **detector de Harris** es efectivo para detectar esquinas pero puede ser menos selectivo que Shi-Tomasi.
- El **detector de Shi-Tomasi** ofrece mejor control sobre la distribución y calidad de las esquinas detectadas.